FuelPrint Operations guide

For Aditya Singhal and Abhivansh · step by step, with the Codex prompts · version 2, 18 Oct 2026

What changed in version 2 for you

- **Automatic steps go through `AutomationBus`.** Your event listeners no longer save records themselves. They submit a command as the `automation` actor: `open_investigation`, `run_analysis`, `propose_mission`, `expire_mission`.
- **The analysis now carries the learned ranker's order** (rank 1, 2, 3) and there is an AI wording record with plain words. OP03 shows both.
- **Missions are proposed automatically** and can be flown by the autopilot. OP04 shows who proposed and who is flying.
- **The board notice is now sent by SMS** to the areas in `notice.area_ids`. OP05 shows what was sent.
- **Abhivansh:** OP01 shows the normal-air model's "unusual air" flags, and he builds a new page, **OP07 AI and automation**.
- **New dates:** the core foundation lands Mon 19 Oct morning, the fake server Mon 19 Oct evening. Aditya Singhal starts with the backend; the screens follow.

What Operations does

Operations is the control room. It watches the four sensor boxes, notices when something unusual is happening, shows which factories could explain it, proposes the next measurement, and lets a supervisor approve it. At the end, a board reviews the case and sends an area-level notice by SMS. Laptop 5 also shows the AI and automation page: what the models learned from, how they tested, and everything the system did by itself.

The thinking (smoke-spread reasoning, the learned ranker, the planner, the normal-air model, the plain wording) is done by the science and AI packages, which Kiro builds. Operations does not do the maths. It decides **when** to ask for an analysis, **saves** exactly what came back, and **shows** it clearly with its limits.

Who does what

Person	Builds	Also
Aditya Singhal	The `ops` backend package and screens OP03, OP04, OP05, OP06, plus the app frame	Stands at laptop 1 during the demo
Abhivansh	Screens OP01 (sensor health), OPHW (hardware) and **OP07 (AI and automation)**	Studies the study pack and the judge questions. Pitches. Stands at laptop 5.

Abhivansh's build work is kept small on purpose. His bigger job is to understand everything well enough to answer any judge, and OP07 is the page he will point at when they ask "where is the AI?".

The tech you will touch, in plain words

Spring Boot is a Java framework that turns your classes into a web server. A `@Service` class holds your logic. The core gives you a `CommandBus`: you register a handler for each of your command types, and the core calls it when that command arrives.

A command handler receives a command (like `approve_mission`), checks it is allowed, and returns new records to save. The core saves them and the event in one database transaction, then sends the receipt. Your handler never sends anything on the socket itself.

An event is the server saying "this record was saved". Your backend can listen to events from other packages, for example the science engine raising a `trigger`. In Spring this is an `@EventListener` method.

The automation actor (v2). When something should happen by itself (an alarm opens an investigation), your listener calls `AutomationBus.submit("open_investigation", payload)`. That sends a normal command as the actor `automation`, through the same checks, with a receipt. In each handler you list which roles may send it; add `automation` only where the agreement's command table says so. Never for `approve_mission`, `reject_mission` or `record_board_decision`.

A revision is a version number on a record. If the screen says "I last saw revision 7" and the record is now at 8, the command is rejected as stale. This stops two laptops overwriting each other.

An interface is a promise of what a class can do, without saying how. You call `Ranker.analyze(...)`, `LearnedRanker.order(...)`, `Planner.plan(...)` and `AlertAreas.downwindOf(...)`. Until Kiro finishes the real ones, the core plugs in stub versions that return fixed answers. Your code does not change when the real ones arrive.

SQLite and Flyway. Your tables are created by SQL files in `server/src/main/resources/db/migration/`. Operations files are numbered `V200__...` to `V299__...` so they never clash with other groups.

React screens get live data from `useLive()` (ticks and events) and the current state from `useSnapshot()`, both from `@fp/shared`. To do something, call `sendCommand(type, payload, expectedRevision)`. It returns the receipt. `AutomationLog` shows what the system did by itself.

What you get from others

From	What	Ready
Core (Kiro)	Server, database, `CommandBus`, `AutomationBus`, `RecordStore`, interfaces and stubs	**Mon 19 Oct morning** (Claude messages the group)
Core (Kiro)	Sessions, `/ws/live`, fake replay mode, `@fp/shared` with `AutomationLog`	**Mon 19 Oct evening**
Science and AI (Kiro)	Real `Detector`, `Ranker`, `LearnedRanker`, `Planner`, normal-air anomalies, AI wording, model versions	Join day, Wed 21 Oct. Stubs before that.
Alerts (Kiro)	`AlertAreas`, SMS sending after your board decision, `notification` records	Join day. Stub and LOG_ONLY before that.
Field (Kiro)	Mission moves through OUTBOUND to COMPLETE, flown by a visitor or the autopilot; `screening_record` events after the capture review	Join day
Cases	`finding` records for the board screen	Join day
Example data	`apps/shared/fixtures/*.json` with one record of every type	Now (version 2)

What you give to others

- `investigation` records and their state changes, including `warned_area_ids`
- `analysis` and `proposal` records exactly as science and AI returned them (through `run_analysis`)
- `mission` records in state PROPOSED and APPROVED (Field picks up APPROVED ones)

- `board_decision` records with `notice.area_ids` (the alerts package sends the SMS)
- The rule that an approved mission expires when the analysis or wind changes

How to use the prompts

1. Open a terminal in the repo folder, on your own branch.
2. Type `codex` and paste the prompt for your step.
3. Codex reads `AGENTS.md` and `contracts/AGREEMENT.md` by itself. You don't need to paste them.
4. When it finishes, run the check in that step yourself. If it fails, paste the error back into Codex.
5. Commit, push, open a pull request.

Read every file Codex changed before you commit. You must be able to explain your screen and your rules to a judge.

Steps for Aditya Singhal

S0 · Get the core (Mon 19 Oct, morning)

```
git pull origin main
git checkout ops/singhal && git merge main
tools/test-all.sh
```

Done when the core tests pass on your laptop. You can start S1 now; the fake server for screens comes in the evening.

S1 · Investigation backend (Mon 19 Oct)

```
Build the investigation workflow in
server/src/main/java/fuelprint/ops.
- Migration V200__ops_investigation.sql for the
investigation record table (revision rows, as in AGREEMENT
section 6), including warned_area_ids.
- @EventListener on trigger events: call
AutomationBus.submit with command open_investigation (role
automation only). The handler creates an investigation in
state OPEN that points to the trigger, and fills
warned_area_ids from AlertAreas.downwindOf(trigger sensor,
current wind). One open investigation per run at a time; a
second trigger during an open one is attached to it, not a
new investigation.
- Command schedule_survey (role analyst): creates an
investigation without a trigger, for the case where no
alarm fired (scenario SC3). Optional anomaly_id when the
analyst schedules it from a normal-air anomaly.
warned_area_ids stays empty for a survey.
- Command run_analysis (roles automation and analyst):
calls Ranker.analyze with the latest windows, then
LearnedRanker.order(input, physicsAnalysis), and saves ONE
analysis record = the physics analysis plus the learned
fields (learned_order, learned_status,
learned_model_version_id, history_counts). Then calls
Planner.plan and saves the proposal. State ANALYSED. Never
change an outcome, a candidate status or the learned
order.
- After open_investigation is applied, submit run_analysis
as automation.
- @EventListener on screening_record events for this
investigation's mission: state EVIDENCE_BACK, then submit
run_analysis as automation again.
- Every listener uses AutomationBus.submit; no listener
saves records itself. Unit tests with the stub
Ranker/LearnedRanker/Planner/AlertAreas: trigger opens
investigation as automation (receipt automatic=true);
second trigger attaches; survey works with no trigger and
warns no area; evidence re-runs analysis; analyst role
required for schedule_survey; run_analysis keeps the
physics outcome and statuses unchanged; repeated
command_id does nothing.
```

Check: `tools/test-all.sh` passes.

S2 · App frame and navigation (Mon 19 Oct, evening)

First start the fake server:

```
git pull origin main && git merge main
tools/start-fake.sh          # leave it running
tools/dev-app.sh operations  # opens
http://localhost:5173/operations/
```

```
Build the Operations app frame in apps/operations. Use
@fp/shared for the live connection, station login,
StaleBanner, Explain and AutomationLog.
- Top bar: run label and state, simulation time, wind
(from-degrees and m/s with an arrow), connection status
(live / stale / offline), logged-in roles.
- Left menu: Health (OP01), Investigation (OP03), Missions
(OP04), Board (OP05), Scenarios (OP06), AI and automation
(OP07), Hardware (OPHW). Use React Router routes under
/operations/.
- A "next action" strip under the top bar that says in one
sentence what the team should do next, based on the
current investigation state from the snapshot. If
automation is about to do it, say so ("Automation will
propose the mission").
- A small "Done automatically" feed (AutomationLog, last 5
items) in the corner of every page.
- Station login screen if there is no session (calls POST
/api/session/station).
- Put placeholder pages for each route; Abhivansh builds
OP01, OP07 and OPHW. Add component tests with Vitest for
the top bar states (live, stale, offline).
```

Check: stop the fake server; within 3 seconds the badge shows "stale" and buttons grey out. Start it again; it comes back by itself.

S3 · Investigation screen OP03 (Tue 20 Oct, morning)

```
Build OP03 at /operations/investigations/:id in
apps/operations.
- Header: what started it (trigger on sensor N2 at time t,
or "scheduled survey"), state, which windows of data were
used, and "opened automatically" when the receipt was
automatic.
- Outcome badge for the analysis outcome with a one-line
meaning from AGREEMENT section 6, and the qualification
(always shown, e.g. "synthetic model only, not field
validated").
- Candidate list: each explanation (single factory, pair,
background, unknown) with its fit number as a bar (lower
fits better; NOT a percentage), status SURVIVES or
REJECTED and the reason. Unknown/unregistered source is
always shown as a row.
- "AI order" column from analysis.learned_order: show the
rank (1, 2, 3) next to each surviving explanation, never
the order_score. Explain line: "The learned ranker's
order, trained on simulated nights. It can't bring back a
rejected factory." Show learned_status if not OK, and link
the model version to OP07. Show history_counts as "fit on
earlier runs: 3".
- "What we ignored": excluded readings with reasons
(stale, flow fault, calibration).
- "What this can't tell you": the limitations list, always
visible.
- Explanation: analysis.explanation_text, and under it the
wording record for this analysis labelled "Plain words by
AI (it cannot change any number)", or "fixed text" if
wording.source is TEMPLATE.
- "Who was warned": the warned areas and the SMOKE_WARNING
notification for this investigation (sent, failed or
suppressed with reason).
- History: earlier analyses for this investigation, so
judges see the ranking change after the drone sample.
Every value has an Explain line. Handle loading, empty,
stale and interrupted.
```

Check: in the fake replay, the ranking shows S01 and S02 surviving first with an AI order, then only S02 after the sample, and the history shows both.

S4 · Mission backend and screen OP04 (Tue 20 Oct)

```
Backend in fuelprint.ops:
- Command propose_mission (roles analyst and automation):
takes an action from the latest proposal (or a survey),
```

```
creates a mission record in state PROPOSED with target,
vehicle, method_id, channel_ids, route from the proposal,
pilot NONE, expiry 300 sim seconds, and dependency_hash =
SHA-256 of (analysis id + revision, proposal id +
revision, wind from-degree rounded to 10, wind speed
rounded to 0.5, route node list). Automation may only
propose best_action_id, and only if
automation_settings.auto_propose is on.
- After run_analysis saves a proposal with result
BEST_ACTION and there is no PROPOSED or APPROVED mission
for this investigation, submit propose_mission as
automation.
- Command approve_mission (supervisor only; refuse
automation): reject if the hash no longer matches the
current state, if expired, or if not PROPOSED. Otherwise
state APPROVED.
- Command reject_mission (supervisor only; refuse
automation) with a reason.
- Command expire_mission (automation only): when a new
analysis is saved or the wind bucket changes, a listener
submits it for any APPROVED mission that has not started;
state EXPIRED with reason NEEDS_NEW_APPROVAL.
- If the proposal result is NO_USEFUL_MEASUREMENT,
propose_mission is rejected with that reason. Tests for
each guard, including a wind change between propose and
approve, automation refused for approve and reject, and
automation proposing only the best action.

Screen OP04 at /operations/missions/:id:
- Card for the proposed action: target point, height,
vehicle, method, channels, full time in minutes (setup,
travel, purge, settle, collect, recover, return), energy
needed, why this point helps (from the proposal), and the
cheaper option side by side. "Proposed automatically" or
"proposed by analyst".
- Rejected actions list with reasons (unreachable, unsafe,
no benefit).
- Approve and Reject buttons for the supervisor role only,
with the expiry countdown. If the proposal says
NO_USEFUL_MEASUREMENT, show that as a proper result: "no
measurement would help right now" and why.
- Live mission state once approved (from ticks/events),
who is flying (pilot: visitor, operator or autopilot), and
"autopilot starts in N s if nobody flies" while waiting.
```

Check: the mission appears proposed automatically; approve works once; approving again with the old revision shows "someone already changed this"; changing the wind in replay makes an approved mission expire automatically.

S5 · Board screen OP05 and board decision (Tue 20 Oct, evening)

```
Backend: command record_board_decision (board only; refuse
automation) in fuelprint.ops. Allowed only when the case
has a finding. decision is one of the four values in
AGREEMENT section 6. notice has area_ids (ids from
contracts/reference/alert-areas.json; default = the
investigation's warned_area_ids), area text, time interval
and status text; it must never contain a source label S01-
S06. The alerts package sends the SMS after it is saved;
you don't send anything. Tests: rejected without a
finding; rejected if notice text contains S0x; rejected if
an area id is unknown; automation refused.

Screen OP05 at /operations/board:
- Case summary: investigation, finding outcome, its
limitations, legal suitability "not assessed".
- Counts across all runs today: supported, rejected,
inconclusive, plus time spent and repeated missions. Show
all three outcomes with the same visual weight.
- Decision form, an area picker (the four areas, warned
ones ticked), and a phone-sized preview of the SMS notice
with the line "This is sent by SMS to people signed up for
these areas".
- After saving: the BOARD_UPDATE notification with sent,
failed or suppressed counts and reason.
```

S6 · Scenario screen OP06 (Wed 21 Oct, morning)

```
Screen OP06 at /operations/scenarios for the presenter
role:
- Pick a preset (SC1, SC2, SC3 from AGREEMENT section 10)
with a one-line description of each, start, pause, resume
and reset buttons.
- Show that reset makes a new run and keeps old cases,
model versions and SMS sign-ups.
- Show the time scale and that captures can run 15x faster
while the vehicle stands still.
- One line linking to OP07 for the automation switches.
- In BLIND mode there are no truth controls. (TEACHING
controls come in the finale.)
```

S7 · Join day (Wed 21 Oct, 2 PM, with everyone)

Run all three scenarios end to end with the real server, once with automation on and once with it off. Write down every mismatch in the group chat with a screenshot. Fix your side, and push by evening.

Steps for Abhivansh

A0 · Set up (Mon 19 Oct, evening)

Same as S2's first block, on branch `ops/abhivansh`.

A1 · Sensor health panel OP01 (Mon 19 Oct evening to Tue 20 Oct)

```
Build OP01 at /operations/health in apps/operations (route
already exists as a placeholder from the app frame).
- One card per sensor box N1 to N4: position, inlet height
(3 m), status, and calibration state.
- For each channel (SO2 ppb, CO2 ppm, particles ug/m3
marked "optical estimate"): current value, unit, quality
flags, age of the last reading, and a 5-minute Recharts
line from live ticks.
- Quality flags explained in plain words (OK, STALE,
FLOW_FAULT, SATURATED, CALIBRATION_EXPIRED). A value with
a bad flag is greyed, never hidden.
- Normal-air flags: for each OPEN anomaly record on a box,
show "Unusual for this hour and wind", the observed mean
against the expected band (low to high), since when, and
"flagged automatically by the normal-air model, trained on
real CPCB Murthal SO2 data". If the anomaly has suggestion
SCHEDULE_SURVEY, show a "Schedule a survey" button for the
analyst role that sends schedule_survey with the
anomaly_id. Explain that an anomaly is a hint, not an
alarm.
- A small map (SVG, top view) with the four boxes, the six
factory outlines S01-S06, roads, the wind arrow, and the
four alert areas from contracts/reference/alert-areas.json
(warned ones highlighted).
- A permanent note: "No alarm does not mean the air is
clean. A plume can pass above or beside every box."
- When a trigger fires, highlight the box and link to the
investigation. Every number gets an Explain line.
Component tests for flag display and for the anomaly card.
```

Check: in replay, N2's SO2 rises, the card highlights, the unusual-air flag appears, and both link onward.

A2 · AI and automation page OP07 (Tue 20 Oct)

```
Build OP07 at /operations/ai in apps/operations. Read-only
for everyone except the buttons named below. Data:
model_version, automation_settings, notification and
alert_subscription records, plus AutomationLog.
1. "Where the AI is": seven short cards (the AI pieces in
AGREEMENT section 11), each with one plain line on what it
does and what it learned from.
2. Model cards, one per model_kind (LEARNED_RANKER,
NORMAL_AIR), for the ACTIVE version: version, trained on
(synthetic nights, case examples, real hours, note), where
trained (Mac CPU or DGX A100), test results as counts
("first in N of M test nights", misses shown with the same
weight), real checks, gate result. Below: the version
history including any REJECTED_BY_GATE version and why.
3. Learning loop: the lessons list (cases used, and cases
skipped with the reason, e.g. "SO2 rejected, not a
lesson"). A "Retrain now" button for presenter and analyst
that sends retrain_model with model_kind LEARNED_RANKER.
After it finishes, show old and new test counts side by
side.
```

```
4. Automation switches (presenter only): one toggle per
setting in automation_settings (auto_propose,
auto_autopilot with autopilot_wait_s, auto_capture,
auto_seal, auto_alerts, auto_retrain), each with one plain
line. Sends change_automation.
5. Automation log: the full AutomationLog, newest first,
with plain words and time.
6. SMS alerts: sign-ups per area (counts only, masked
numbers on hover for the presenter), notifications (kind,
areas, text, sent, failed, suppressed and why). "Send test
SMS" (presenter, send_alert kind TEST) and "Delete all
phone numbers now" (presenter, purge_subscriptions, with a
confirm step). Never show a percentage or a chance
anywhere on this page. Every number gets an Explain line.
Component tests for the model card and the switches.
```

Check: with the fixtures, both model cards show, the toggles send `change_automation`, and no `%` sign appears on the page.

A3 · Hardware page OPHW (Tue 20 Oct, evening)

```
Build a static page at /operations/hardware in
apps/operations using contracts/reference/hardware-
selection.json and the text in docs/plan/05-study-pack.md
(hardware section). Sections:
1. "What is simulated today": everything in the demo is
synthetic.
2. Three hardware paths as cards: (a) optional bench box
under Rs 2,000, ESP32 + Sharp optical dust sensor,
relative dust only; (b) first real test with a rented
reference SO2 analyser (Teledyne API N100); (c) later
custom node with Alphasense SO2-B4, Vaisala GMP252 CO2,
Sensirion SPS30 particles, ESP32-S3, ADS1220 ADC, two
pumps and two flow meters.
3. A block diagram (SVG) of the custom node: separate gas
paths and particle path, pumps after the sensors, flow
meters, power tree 24 V -> 12/5/3.3 V.
4. Cost stages A to F as a table with "unquoted allowance"
labels.
5. "What is not tested yet": no physical test has been
done. No live data on this page. Keep text short and
plain.
```

A4 · Study and pitch (every day)

Follow the schedule at the end of the study pack. Practise the 2-minute pitch on Monday, the full 7-minute demo talk at Wednesday's rehearsal.

Finale work (25 to 29 Oct)

Item	Who
OP02 placement and coverage: move a box, see which sectors and source pairs it can tell apart, "no adequate layout" as a real answer	Aditya Singhal
TEACHING mode controls: change wind, stack height, block a branch, with a log of each change	Aditya Singhal
More scenarios on OP06	Aditya Singhal
OP07: compare learned ranker version 1 (Mac) and version 2 (DGX) side by side; the real tracer test results	Abhivansh
Hardware page polish and a printed one-page hardware sheet for judges	Abhivansh

Done means

- Your tests pass and your pull request is merged.
- All three scenarios work on your screens with the real server, with automation on and off.
- A teammate who has not seen your screen can say what every number means.
- You can explain to a judge why the screen says what it says, and which steps the system did by itself.